

Functional API

`mod` func

FUNCTION	DESCRIPTION
<code>task</code>	Define a LangGraph task using the <code>task</code> decorator.

`class` endpoint

Bases: `Generic[ContextT]`

Define a LangGraph workflow using the `endpoint` decorator.

Function signature

The decorated function must accept a **single parameter**, which serves as the input to the function. This input parameter can be of any type. Use a dictionary to pass **multiple parameters** to the function.

Injectable parameters

The decorated function can request access to additional parameters that **will** be injected automatically at run time. These parameters include:

Parameter	Description
<code>config</code>	A configuration object (aka <code>RunnableConfig</code>) that holds run-time configuration values.
<code>previous</code>	The previous return value for the given thread (available only when a checkpointer is provided).
<code>runtime</code>	A <code>Runtime</code> object that contains information about the current run, including context, store, writer

The entrypoint decorator can be applied to sync functions or async functions.

State management

The `previous` parameter can be used to access the return value of the previous invocation of the entrypoint on the same thread id. This value is only available when a checkpointer is provided.

If you want `previous` to be different from the return value, you can use the `entrypoint.final` object to return a value while saving a different value to the checkpoint.

PARAMETER	DESCRIPTION
<code>checkpointer</code>	Specify a checkpointer to create a workflow that can persist its state across runs. TYPE: <code>BaseCheckpointSaver</code> <code>None</code> DEFAULT: <code>None</code>
<code>store</code>	A generalized key-value store. Some implementations may support semantic search capabilities through an optional <code>index</code> configuration. TYPE: <code>BaseStore</code> <code>None</code> DEFAULT: <code>None</code>
<code>cache</code>	A cache to use for caching the results of the workflow. TYPE: <code>BaseCache</code> <code>None</code> DEFAULT: <code>None</code>
<code>context_schema</code>	Specifies the schema for the context object that will be passed to the workflow. TYPE: <code>type[ContextT]</code> <code>None</code> DEFAULT: <code>None</code>
<code>cache_policy</code>	A cache policy to use for caching the results of the workflow. TYPE: <code>CachePolicy</code> <code>None</code> DEFAULT: <code>None</code>
<code>retry_policy</code>	A retry policy (or list of policies) to use for the workflow in case of a failure. TYPE: <code>RetryPolicy</code> <code>Sequence[RetryPolicy]</code> <code>None</code> DEFAULT: <code>None</code>

**`config_schema` Deprecated**

The `config_schema` parameter is deprecated in v0.6.0 and support will be removed in v2.0.0. Please use `context_schema` instead to specify the schema for run-scoped context.

Using endpoint and tasks

```
import time

from langgraph.func import entrypoint, task
from langgraph.types import interrupt, Command
from langgraph.checkpoint.memory import InMemorySaver

@task
def compose_essay(topic: str) -> str:
    time.sleep(1.0) # Simulate slow operation
    return f"An essay about {topic}"

@entrypoint(checkpointer=InMemorySaver())
def review_workflow(topic: str) -> dict:
    """Manages the workflow for generating and reviewing an essay.

    The workflow includes:
    1. Generating an essay about the given topic.
    2. Interrupting the workflow for human review of the generated essay.

    Upon resuming the workflow, compose_essay task will not be re-executed
    as its result is cached by the checkpointer.

    Args:
        topic: The subject of the essay.

    Returns:
        dict: A dictionary containing the generated essay and the human review.
    """
    essay_future = compose_essay(topic)
    essay = essay_future.result()
    human_review = interrupt({
        "question": "Please provide a review",
        "essay": essay
    })
    return {
        "essay": essay,
        "review": human_review,
    }

# Example configuration for the workflow
config = {
    "configurable": {
        "thread_id": "some_thread"
    }
}

# Topic for the essay
topic = "cats"

# Stream the workflow to generate the essay and await human review
for result in review_workflow.stream(topic, config):
    print(result)
```

```
# Example human review provided after the interrupt
human_review = "This essay is great."

# Resume the workflow with the provided human review
for result in review_workflow.stream(Command(resume=human_review), config):
    print(result)
```

Accessing the previous return value

When a checkpoint is enabled the function can access the previous return value of the previous invocation on the same thread id.

```
from typing import Optional

from langgraph.checkpoint.memory import MemorySaver

from langgraph.func import entrypoint

@entrypoint(checkpointer=InMemorySaver())
def my_workflow(input_data: str, previous: Optional[str] = None) -> str:
    return "world"

config = {"configurable": {"thread_id": "some_thread"}}
my_workflow.invoke("hello", config)
```

Using `entrypoint.final` to save a value

The `entrypoint.final` object allows you to return a value while saving a different value to the checkpoint. This value will be accessible in the next invocation of the entrypoint via the `previous` parameter, as long as the same thread id is used.

```
from typing import Any

from langgraph.checkpoint.memory import MemorySaver

from langgraph.func import entrypoint

@entrypoint(checkpointer=InMemorySaver())
def my_workflow(
    number: int,
    *,
    previous: Any = None,
) -> entrypoint.final[int, int]:
    previous = previous or 0
    # This will return the previous value to the caller, saving
    # 2 * number to the checkpoint, which will be used in the next invocation
    # for the `previous` parameter.
    return entrypoint.final(value=previous, save=2 * number)

config = {"configurable": {"thread_id": "some_thread"}}

my_workflow.invoke(3, config) # 0 (previous was None)
my_workflow.invoke(1, config) # 6 (previous was 3 * 2 from the previous
invocation)
```

METHOD	DESCRIPTION
<code>__init__</code>	Initialize the entrypoint decorator.
<code>__call__</code>	Convert a function into a Pregel graph.

`class final` dataclass

Bases: `Generic[R, S]`

A primitive that can be returned from an entrypoint.

This primitive allows to save a value to the checkpointer distinct from the return value from the entrypoint.

Decoupling the return value and the save value

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.func import entrypoint

@entrypoint(checkpointer=InMemorySaver())
def my_workflow(
    number: int,
    *,
    previous: Any = None,
) -> entrypoint.final[int, int]:
    previous = previous or 0
    # This will return the previous value to the caller, saving
    # 2 * number to the checkpoint, which will be used in the next invocation
    # for the `previous` parameter.
    return entrypoint.final(value=previous, save=2 * number)

config = {"configurable": {"thread_id": "1"}}

my_workflow.invoke(3, config) # 0 (previous was None)
my_workflow.invoke(1, config) # 6 (previous was 3 * 2 from the previous
invocation)
```

attr **value** instance-attribute

value: R

Value to return. A value will always be returned even if it is `None`.

attr **save** instance-attribute

save: S

The value for the state for the next checkpoint.

A value will always be saved even if it is `None`.

meth **__init__**

```
__init__(
    checkpointer: BaseCheckpointSaver | None = None,
    store: BaseStore | None = None,
    cache: BaseCache | None = None,
    context_schema: type[ContextT] | None = None,
    cache_policy: CachePolicy | None = None,
    retry_policy: RetryPolicy | Sequence[RetryPolicy] | None = None,
```

```

    **kwargs: Unpack[DeprecatedKwargs],
) -> None

```

Initialize the entrypoint decorator.

meth `__call__`

```

__call__(func: Callable[..., Any]) -> Pregel

```

Convert a function into a Pregel graph.

PARAMETER	DESCRIPTION
<code>func</code>	The function to convert. Support both sync and async functions. TYPE: <code>Callable[..., Any]</code>
RETURNS	DESCRIPTION
<code>Pregel</code>	A Pregel graph.

func `task`

```

task(
    __func_or_none__: Callable[P, Awaitable[T]] | Callable[P, T] | None = None,
    *,
    name: str | None = None,
    retry_policy: RetryPolicy | Sequence[RetryPolicy] | None = None,
    cache_policy: CachePolicy[Callable[P, str | bytes]] | None = None,
    **kwargs: Unpack[DeprecatedKwargs],
) -> (
    Callable[[Callable[P, Awaitable[T]] | Callable[P, T]], _TaskFunction[P, T]]
    | _TaskFunction[P, T]
)

```

Define a LangGraph task using the `task` decorator.

 Requires python 3.11 or higher for async functions

The `task` decorator supports both sync and async functions. To use async functions, ensure that you are using Python 3.11 or higher.

Tasks can only be called from within an `entrypoint` or from within a `StateGraph`. A task can be called like a regular function with the following differences:

- When a checkpointer is enabled, the function inputs and outputs must be serializable.
- The decorated function can only be called from within an `entrypoint` or `StateGraph`.
- Calling the function produces a future. This makes it easy to parallelize tasks.

PARAMETER	DESCRIPTION
<code>name</code>	An optional name for the task. If not provided, the function name will be used. TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>
<code>retry_policy</code>	An optional retry policy (or list of policies) to use for the task in case of a failure. TYPE: <code>RetryPolicy</code> <code>Sequence[RetryPolicy]</code> <code>None</code> DEFAULT: <code>None</code>
<code>cache_policy</code>	An optional cache policy to use for the task. This allows caching of the task results. TYPE: <code>CachePolicy[Callable[P, str bytes]]</code> <code>None</code> DEFAULT: <code>None</code>

RETURNS	DESCRIPTION
<code>Callable[[Callable[P, Awaitable[T]] Callable[P, T]], _TaskFunction[P, T]] _TaskFunction[P, T]</code>	A callable function when used as a decorator.



Sync Task



```
from langgraph.func import entrypoint, task

@task
def add_one_task(a: int) -> int:
    return a + 1

@entrypoint()
def add_one(numbers: list[int]) -> list[int]:
    futures = [add_one_task(n) for n in numbers]
    results = [f.result() for f in futures]
    return results

# Call the entrypoint
add_one.invoke([1, 2, 3]) # Returns [2, 3, 4]
```



Async Task



```
import asyncio
from langgraph.func import entrypoint, task

@task
async def add_one_task(a: int) -> int:
    return a + 1

@entrypoint()
async def add_one(numbers: list[int]) -> list[int]:
    futures = [add_one_task(n) for n in numbers]
    return asyncio.gather(*futures)

# Call the entrypoint
await add_one.ainvoke([1, 2, 3]) # Returns [2, 3, 4]
```